A Project Report

On

Continuum: A Durable Execution Engine and Reliability Gateway for LLM-Powered Applications

For the Degree of

Bachelor of Technology

In

Information Technology

Submitted by

PRN	Name
246200002	Abhay Vishwanath Patil
246200005	Suraj Sanjay Chavan

Under the Guidance of

[Guide's name]



Department of Information Technology

Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute)

AY 2026 - 27

Acknowledgement

With a deep sense of gratitude, we would like to thank all the people who have lit our path with their kind guidance. We are very grateful to these intellectuals who did their best to help during our project work.

The special gratitude goes to our guide, [Guide's name], for the valuable guidance, constant encouragement and suggestions that shaped this project. We remain in debt for the consistent instructions and support given throughout its completion.

We are thankful to the Head of the Department and to all the faculty members and technical staff of the Information Technology Department for their extensive, excellent and precious guidance in the completion of this work. We thank all our class colleagues for their appreciable help with our working project.

Finally, we thank our families for their patience and support, and the friends who tried the console and the sample applications and told us honestly where they were confusing.

Vision, Mission & Course Outcomes

VISION:

To be an Information Technology programme as the first choice by the aspiring students and prospective employers by implementing world class education practices

MISSION:

To meet above Vision, all stakeholders of this programme are committed towards Outcome Based Education Philosophy.

Under-Graduate POs

- **PO1: Engineering Knowledge:** Apply the knowledge of mathematics, engineering fundamentals and computational science to the solution of engineering problems.
- **PO2: Problem Analysis:** Identify, formulate, interpret and analyze complex engineering problems leading to substantiated conclusions.
- **PO3: Design/Development of Solutions:** Design systems, components or processes to meet desired needs within realistic constraints such as economic, environmental, societal and ethical considerations.
- **PO4: Conduct Investigations of Complex Problems:** Use research-based knowledge and methods including design of experiments, analysis, interpretation and synthesis of information to provide valid conclusions.
- **PO5: Modern Tool Usage:** Select and apply appropriate techniques, engineering skills and modern IT tools to prototype the model of complex engineering activities.
- **PO6: The Engineer and Society:** Apply contextual knowledge pertaining to social, secure, legal and cultural issues with consequent responsibilities relevant to Information Technology.
- **PO7: Environment And Sustainability:** Understand the impact of professional engineering solutions in social, environmental and the global contexts, demonstrating the knowledge of, and the need for sustainable development.
- **PO8: Ethics:** Apply ethical principles and commit to professional ethics with responsibilities and norms of the engineering practices.
- **PO9: Individual and Teamwork:** Work effectively as an individual as well as a member or a leader in diverse teams for multidisciplinary settings.
- **PO10: Communication:** Communicate effectively with the engineering community and with society at large, such as, being able to comprehend and write reports and design documentation to make effective presentations.
- **PO11: Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply those to original work or contemporary issues, as a member or/and a leader in a team or an entrepreneur, to manage projects in multidisciplinary environments

Declaration

We hereby declare that the work presented in this project report titled “CONTINUUM: A DURABLE EXECUTION ENGINE AND RELIABILITY GATEWAY FOR LLM-POWERED APPLICATIONS” submitted by us in the partial fulfilment of the requirement of the award of the degree of Bachelor of Technology (B.Tech) submitted in the Department of Information Technology, Walchand College of Engineering, Sangli, is an authentic record of our project work carried out under the guidance of [Guide's name].

Place: Sangli.
Date: /05/2027

PRN	**Name**
246200002	Abhay Vishwanath Patil
246200005	Suraj Sanjay Chavan

Table of Contents

Sr. no.	Name of the Content	Page No.
i	Abbreviations	i
ii	List of Figures	ii
iii	List of Tables	iii
iv	Abstract	iv
1	Introduction	1
2	Literature Survey	5
3	Methodology	7
4	Result and Discussion	24
5	Conclusion	30
6	References	31

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
SDK	Software Development Kit
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
SQL	Structured Query Language
JDBC	Java Database Connectivity
JPA	Jakarta Persistence API
PII	Personally Identifiable Information
BYOK	Bring Your Own Key
AES-GCM	Advanced Encryption Standard, Galois/Counter Mode
UI	User Interface
SPA	Single Page Application
DAG	Directed Acyclic Graph
EDF	Earliest Deadline First
OCR	Optical Character Recognition
MMU	Memory Management Unit (context optimizer)
RAG	Retrieval-Augmented Generation
p50 / p95 / p99	50th / 95th / 99th percentile latency
UUID	Universally Unique Identifier
ER	Entity Relationship
PRN	Permanent Registration Number

List of Figures

Sr No.	Content	Page No.
Fig. 3.3.4	Use Case Diagram	10
Fig. 3.3.6	Entity-Relationship Diagram	11
Fig. 3.4.1	Proposed System Architecture	12
Fig. 3.4.2	Request Pipeline of the AI Gateway	13
Fig. 3.4.3	Evidence Pipeline with Specialist Models	13
Fig. 3.4.4	Sequence Diagram: Workflow Step, Crash and Recovery	14
Fig. 3.4.5	Event Log and Deterministic Replay	14
Fig. 3.5.1	Proposed System Flowchart (one gateway request)	15
Fig. 3.7.1	API Keys, Provider Keys and Webhooks	18
Fig. 3.7.2	Command Centre	18
Fig. 3.7.3	Gateway: Requests, Latency, Models and Failures	19
Fig. 3.7.4	Routing: Dispatch Network and Provider Statistics	19
Fig. 3.7.5	Durable Workflow Run History	20
Fig. 3.7.6	Run Timeline with Retries and Findings	20
Fig. 3.7.7	Saga Compensation of a Cancelled Run	21
Fig. 3.7.8	Semantic Cache: Hit Rate, Tokens Saved and Threshold	21
Fig. 3.7.9	Prompt Guard: Redaction and Injection Screening	22
Fig. 3.7.10	Specialist Catalogue for Evidence Pipelines	23
Fig. 4.1.1	Verification Engine: Answers Solved and Verified as a Graph	24
Fig. 4.1.2	Chaos Lab: Model Failures Injected and Survived	25
Fig. 4.1.3	Autopilot: Policy Optimisation with Canary and Rollback	25
Fig. 4.2.1	Provider Failures Absorbed vs Failures that Reached the Application	26
Fig. 4.2.2	Tokens Billed with the Semantic Cache Off and On	27
Fig. 4.2.3	Response Latency with the Semantic Cache Off and On	27
Fig. 4.3.1	Resolutions Applied over 300 Code Changes	28

List of Tables

Table No.	Title	Page No.
Table 3.2.1	System Requirements	9
Table 3.3.5	SWOT Analysis	11
Table 4.2.1	Benchmark Results (100 requests per arm)	26
Table 4.3.1	Paradox-Healing Benchmark (300 seeded trials)	28
Table 4.4.1	Measurements Recorded during Development	28
Table 4.5.1	Automated Test Suites	29

Abstract

Applications built on large language models inherit the weaknesses of the models and of the services that host them: providers go down or throttle, models state wrong facts with confidence, long multi-step jobs are lost or repeated when a server crashes, and every token is billed. This project presents **Continuum**, a self-hostable platform that sits between an application and its model providers and treats the model as one more unreliable dependency.

An application integrates once, through an OpenAI-compatible endpoint, and gains provider failover and measured routing, a semantic cache, a model cascade, a prompt firewall, answer checks with measured confidence, and evidence pipelines that run specialist vision, speech or OCR models before the language model. Multi-step work runs as durable workflows: every step is recorded as an event in PostgreSQL, a crashed worker’s task is reclaimed and replayed deterministically, side effects leave through a transactional outbox exactly once, and failed runs can be compensated. The system is one Java 21 / Spring Boot 3.3 service with a PostgreSQL 16 database (95 tables, 55 migrations) and a React 18 console that shows every request, decision and cost.

On a seeded benchmark of 100 requests with 25% and 50% of provider calls made to fail, every request was answered and no failure reached the application (22 and 48 failures absorbed); the semantic cache cut tokens billed by 57% and median latency from 34.1 ms to 13.1 ms. Across 300 code changes deployed under running workflows, the engine recorded no crash and no repeated side effect. The platform is covered by 881 unit, 75 integration and 33 console tests, and two sample applications show how an ordinary project adopts it.

1 | Introduction

1.1 Introduction

Large language models (LLMs) are now built into customer-support desks, health and finance assistants, tutoring tools and internal automation. An application usually reaches the model through a hosted provider such as Google Gemini or Groq, sends a prompt, and shows whatever comes back. That simple arrangement hides several kinds of failure: the provider may be down, slow or rate-limited; the model may state something false with complete confidence; a long job made of several model calls and side effects (a refund, an e-mail, a booking) may crash halfway and then be repeated; and every token sent and received is billed.

This project proposes **Continuum**, a platform that sits between an application and its model providers and treats the model as one more unreliable dependency. The application talks to Continuum through an OpenAI-compatible endpoint, so existing code needs only a new address and key. Behind that endpoint Continuum routes, retries and fails over between providers; checks and measures answers; hides personal data; reuses earlier answers; and records why every decision was taken. Work that spans several steps runs as a **durable workflow**, so a crash resumes from the last recorded step instead of starting again.

Continuum does not try to make a model smarter. Its aim is narrower and easier to verify: an application built on it should keep answering when a provider fails, should never repeat a side effect, should say how sure an answer is, and should cost no more than it has to — and all of this should be visible in one console.

1.2 Problem Statement

The purpose of this project was to design and build a self-hostable platform that makes LLM-powered applications reliable, safe and affordable without changing how those applications are written. Instead of every team re-implementing retries, fallbacks, caching, redaction and crash recovery around its model calls, the platform provides them once, behind a standard interface, with every action recorded.

Plain retries are not enough. Retrying a call that already caused a payment pays twice; retrying against the same failing provider fails again; and a retried hallucination is still a hallucination. The platform therefore has to remember what has already happened, choose a different route when one fails, and judge the answer before the application sees it.

1.3 Motivation

The motivation came from how ordinary these failures are. Hosted model APIs have outages and rate limits; free tiers, which student and start-up projects depend on, are the first to be throttled. Models change behind the same name, so an application that worked last month can quietly answer differently today. And when an AI agent performs several steps, a crash in the middle leaves the work half done, with no record of which steps already took effect.

Mature answers to the crash problem exist in distributed systems — event sourcing, durable execution and sagas — and recent research offers ways to measure a model's confidence and to route between cheap and strong models. These ideas are rarely combined for the developer who simply wants an AI feature to work. Continuum brings them together in one service and measures, with a reproducible benchmark, how much they help.

1.4 Contributions of work

The main contribution is Continuum, an end-to-end system that combines durable execution with an AI-aware reliability gateway, together with a console, a benchmark harness and two sample applications.

1.4.1 Durable execution engine

Every workflow step is appended to an event log in PostgreSQL. State is rebuilt by replaying the log deterministically, workers claim work with `FOR UPDATE SKIP LOCKED` and a visibility timeout, and side effects leave through a transactional outbox keyed by an idempotency key, so they happen exactly once. Failed runs can be compensated (saga) and old runs keep replaying after the workflow code changes.

1.4.2 OpenAI-compatible reliability gateway

A single endpoint, `/v1/chat/completions`, routes each request using measured cost, latency and success rate, fails over across providers and models, can race a second provider when the first is slow, and admits, prioritises or refuses requests by deadline and budget.

1.4.3 Answer assurance

Answers can be scored by a quality gate and repaired once, sampled several times to measure agreement in meaning (confidence), and checked against evidence. A model whose answers degrade is taken out of rotation by a semantic circuit breaker, and an agent going round in circles is detected.

1.4.4 Cost control

A semantic cache answers questions already answered in different words, a model cascade asks a cheap model first and escalates only when a judge is not satisfied, prompts are compressed within a quality budget, and cost limits reserve spend before a call is made.

1.4.5 Security and privacy

A prompt firewall removes personal data before it leaves and secrets from answers, and screens for prompt injection. Continuum API keys are stored as hashes; a developer's own provider keys are encrypted with AES-256-GCM and decrypted only at the moment of a call.

1.4.6 Evidence pipelines with specialist models

A pipeline runs purpose-built models (vision, speech, OCR — for example a Roboflow injury detector) before the language model, turns their findings into evidence with stated confidence, applies a confidence policy, and verifies the answer against that evidence. Every step is stored as a trace.

1.4.7 Observability and self-testing

A React console shows every request, decision and cost; decision provenance records why each answer happened. A chaos lab injects infrastructure and model failures on purpose, and an optional Autopilot tunes routing policy but only after replaying past traffic to prove the change is better.

1.5 Objectives

1.5.1 Reliability: To keep answering when a provider fails, so that no provider failure reaches the application while any alternative remains.

1.5.2 Crash safety: To make multi-step AI work survive crashes and restarts, with every side effect performed exactly once.

1.5.3 Trustworthy answers: To measure and report how sure an answer is, check it against its request and its evidence, and flag or repair it instead of passing it on silently.

1.5.4 Lower cost: To reduce tokens billed through caching, cascading and compression, and to keep spend within a budget.

1.5.5 Privacy and security: To keep personal data and secrets out of prompts and answers and to store every credential safely.

1.5.6 Easy integration: To let any application adopt the platform by changing one address, through an OpenAI-compatible API, with no provider SDKs.

1.5.7 Transparency and evidence: To record every decision, expose it in a console, and support the claims above with a reproducible benchmark and automated tests.

2 | Literature Survey

The work this project builds on falls into four groups: durable execution and transaction patterns from distributed systems; the reliability problems of language models; techniques for routing, cascading and caching model calls; and the security and testing practices needed to run such a system.

Durable execution engines such as Temporal and Cadence run a workflow as code whose every step is recorded, so that a crash resumes instead of restarting [1]. The idea rests on **event sourcing**, in which the state of a system is the result of replaying an append-only log of events rather than a value kept in memory [2]. For work that cannot be rolled back by a database transaction, Garcia-Molina and Salem proposed **sagas**: a long transaction split into steps, each with a compensating action that undoes it if a later step fails [3]. When a step must both update a database and send a message, the **transactional outbox** writes the message in the same transaction and dispatches it later with an idempotency key, giving exactly-once effects [4]. Continuum's engine applies these patterns, and claims work with PostgreSQL's `FOR UPDATE SKIP LOCKED` so that many workers can share one queue safely [19].

Language models are known to produce fluent but unsupported statements, a behaviour surveyed by Ji et al. [5]. Kuhn, Gal and Farquhar showed that sampling several answers and measuring how many distinct **meanings** they express (semantic entropy) estimates uncertainty better than comparing their wording [6], and later used it to detect hallucinations [7]. Chen, Zaharia and Zou observed that the behaviour of a hosted model can change substantially over a few months under the same name [9], which motivates watching answer quality continuously rather than only checking for errors. Grounding a model in retrieved or measured evidence, as in retrieval-augmented generation, reduces unsupported claims and keeps the source inspectable [18]; Continuum's evidence pipelines follow the same principle with specialist models instead of a text index.

Cost and latency have their own literature. FrugalGPT showed that a cascade — trying a cheap model first and escalating only when a scorer is not satisfied — can match a strong model's quality at a fraction of the cost [8], and RouteLLM learns which queries need the stronger model [12]. Dean and Barroso's analysis of tail latency introduced **hedged requests**, sending a second request when the first is slower than usual [10]. GPTCache demonstrated semantic caching of model answers by meaning rather than exact text [15]; LLMLingua compresses prompts while keeping their information [14]; and MemGPT pages context in and out of a limited window like an operating system [13]. Choosing among providers whose performance drifts is a bandit problem, for which contextual bandits are a standard approach [17].

On the security side, Greshake et al. showed how instructions hidden in content can hijack an LLM-integrated application (indirect prompt injection) [11], which is why Continuum screens prompts and strips secrets from answers. Chaos engineering, as practised at Netflix, argues that resilience must be tested by injecting failures deliberately and measuring the system's behaviour [16]; Continuum's chaos lab and benchmark harness follow that discipline. The system itself is built with Spring Boot [20], PostgreSQL [19], React [21] and Docker [23], and exposes the OpenAI chat-completions format [22].

Nevertheless, these pieces are usually adopted separately. Workflow engines do not understand model answers, LLM gateways do not make multi-step work crash-safe, and research prototypes for confidence or cascading are not packaged for an ordinary application. Combining durable execution with AI-specific routing, measurement and safety, behind one standard API and with every decision recorded, is the gap this project addresses.

3 | Methodology

3.1 Steps for Implementation

3.1.1 Request intake and authentication:

Every call arrives with a Continuum API key (applications) or a console session (developers). Keys are looked up by prefix and compared as hashes; each key belongs to an app, so traffic and data stay separated between applications of the same account.

3.1.2 Traffic control:

Before any work is done, admission control checks provider capacity, the request's priority and deadline, and the account's cost limit. A request that cannot finish in time or within budget is refused at once with the reason, instead of failing late.

3.1.3 Prompt preparation:

The prompt firewall redacts personal data and screens for injection; context transformers turn logs, tables or e-mail threads into a compact form; the semantic cache returns an earlier answer to the same question; memory, paging and compression prepare the final context.

3.1.4 Routing and failover:

A provider and model chain is chosen from measured cost, latency and success rate, skipping models that the semantic breaker has taken out of rotation. A cascade or hedge may answer first; otherwise the chain is tried in order until one provider answers.

3.1.5 Answer assurance and recording:

The answer can be scored and repaired, its confidence measured, and secrets removed. The request, its cost, the provider that answered, the failovers and every decision are recorded, and the response tells the application which features acted on it.

3.1.6 Durable execution of multi-step work:

Multi-step jobs run as workflows: each step is recorded before the next is scheduled, workers claim tasks safely, side effects go through the outbox, and a failed run can be compensated.

3.2 Methodology

3.2.1 Define Core Functionalities and Requirements:

The key functions are: sign-up, apps and API keys; an OpenAI-compatible chat endpoint with routing and failover; durable workflows with replay, outbox and compensation; per-account switches for each reliability feature; evidence pipelines with specialist models;

memory; a console with live metrics and traces; chaos injection; and a benchmark that measures the claims.

3.2.2 Select Backend and Frontend Technologies:

Backend: Java 21 with Spring Boot 3.3.5, Spring Web, Spring Data JPA and Flyway; one service that hosts the gateway, the workflow engine and the console API.

Database: PostgreSQL 16 for everything: event store, task queues, outbox, request log, model registry, memory, cache and traces (95 tables created by 55 versioned migrations).

Frontend: React 18 with TypeScript 5.6, Vite 5, Tailwind CSS 3 and Motion; a single-page console.

Model providers: Google Gemini and Groq over HTTPS (free tiers supported), or the developer's own keys; specialist providers Roboflow, Hugging Face, Google Vision, Deepgram, AssemblyAI, OCR.space or any HTTP endpoint.

Deployment: Docker Compose for local and single-server use; configuration for Render (backend) and Vercel (console) is included.

3.2.3 Development of APIs and Engines:

Applications use `/v1/chat/completions` and `/api/gateway/*` (chat, pipelines, memory, workflows, feedback) with an API key. The console uses `/api/portal/*` with a session. Every gateway stage is a separate component with its own per-account switch, so a stage that is off passes the request through unchanged.

3.2.4 Workflow Engine and Data Design:

Workflows are defined in code or declaratively (JSON specification of HTTP and model steps). The engine keeps no state in memory between steps: instances, events, decision and activity tasks and outbox rows are database rows, so any worker can continue any workflow.

3.2.5 User Interface and Visualization:

The console has a command centre, a page per feature (with a switch, live counters and a chart or table), run timelines with findings, decision traces, and a developer portal for keys, provider credentials, apps and webhooks.

3.2.6 Testing and Performance Evaluation:

JUnit 5 unit tests cover each component, integration tests run against a real PostgreSQL database, Vitest covers the console, and a benchmark harness replays a seeded workload with injected provider failures and compares the results with the cache off and on.

3.2.7 System Deployment:

The platform runs as three containers (database, service, console) started with `docker compose up`. Secrets such as provider keys and the master encryption key are supplied through environment variables and are never committed.

Category	Requirement
Hardware (recommended)	Dual-core or better CPU, 8 GB RAM, 5 GB free disk
Operating system	Windows 10/11, Linux or macOS; Docker Desktop / Docker Engine with Compose v2
Languages and runtimes	Java 21 (service), Node.js 20 (console build), Python 3.8+ (sample apps, seed and benchmark scripts)
Frameworks	Spring Boot 3.3, Spring Data JPA, Flyway; React 18, Vite 5, Tailwind CSS 3
Database	PostgreSQL 16
AI providers	Google Gemini and/or Groq API key (free tiers are enough); optional specialist providers
Network	Internet access to the model providers; optional webhook receiver
Client	Any modern web browser; any HTTP client or OpenAI-compatible SDK

Table 3.2.1 : System Requirements

3.3 System Analysis and Design

3.3.1 Requirement Specification:

Functional: issue and revoke API keys per app; send chat requests through the gateway; start, signal and inspect workflows; upload files to pipelines; switch each feature on or off; view requests, costs, traces and decisions; inject failures; receive webhooks. **Non-functional:** no provider failure may reach the caller while an alternative exists; no side effect may run twice; credentials must never be stored or logged in clear; every feature must be off by default and must not change behaviour when off; the system must run from a single command.

3.3.2 Feasibility Study:

The project is technically feasible because every component is open source and widely used: Java, Spring Boot, PostgreSQL, React and Docker. The only paid dependency, the model provider, has free tiers (Gemini and Groq), and the failover chain keeps the system working when one of them throttles. Running costs are therefore limited to a small server; the database is the only stateful part.

3.3.3 Data and Workloads:

Continuum does not train a model, so it needs no training dataset. The data it works with is its own telemetry: requests, latencies, token counts, costs, failovers and decisions, which also feed routing and Autopilot. For evaluation, a seeded workload generator

produces repeatable request streams (fixed seed, fixed share of repeated questions) and a chaos injector fails a fixed share of provider calls, so every experiment can be re-run and compared.

3.3.4 Use Case Diagram:

The developer configures the platform, the client application sends traffic, and the operator manages accounts. Continuum in turn uses external model providers, specialist providers and the developer's webhook receiver.

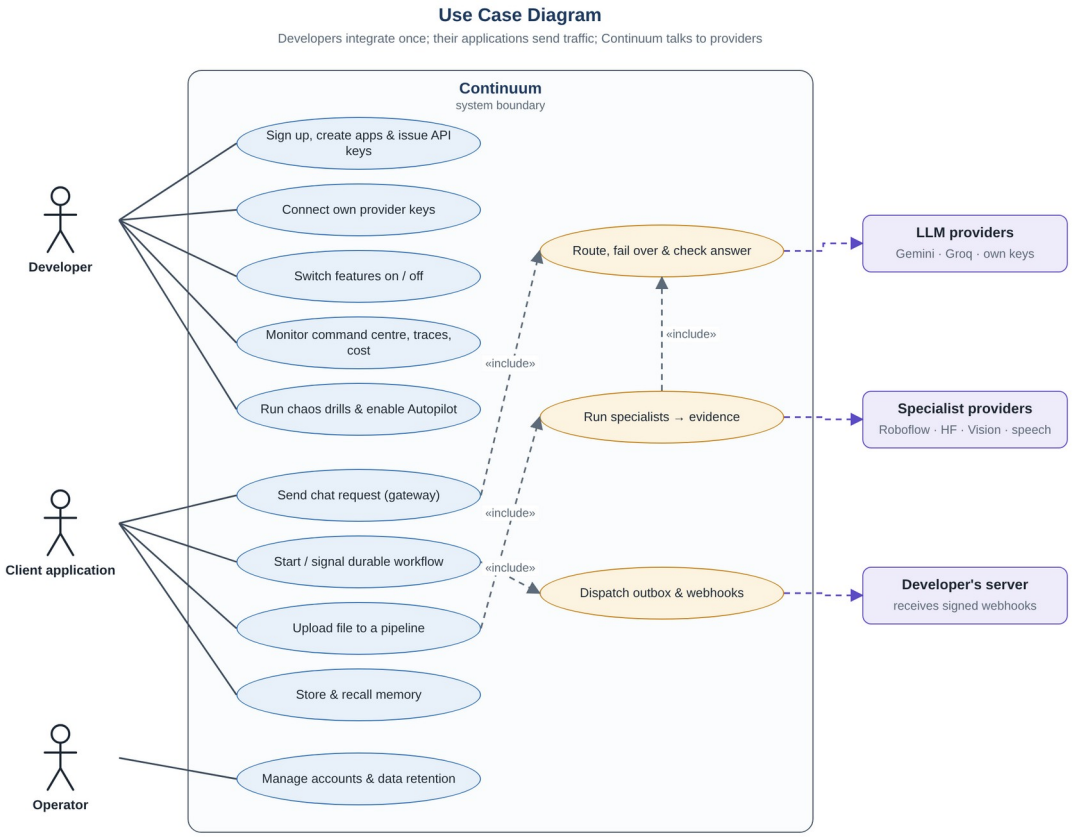


Figure 3.3.4: Use Case Diagram

3.3.5 SWOT Analysis:

Continuum's strength is that one integration brings many protections, each measurable; its weakness is the configuration surface and the dependence on external providers for the answers themselves.

Strengths	Weaknesses	Opportunities	Threats
One OpenAI-compatible endpoint brings failover, caching, checks and recording.	Many features mean many settings to understand.	A hosted multi-tenant service for student and start-up teams.	Provider APIs, model names and prices change often.
Crash-safe workflows with exactly-once side effects.	One service and one database: scales vertically first.	More providers, specialist models and Indian-language apps.	Mature workflow engines and LLM gateways already exist.

Strengths	Weaknesses	Opportunities	Threats
Every decision recorded and visible in the console.	Answers still come from external models.	Streaming tool calls and agent frameworks on top of the engine.	A misconfigured feature could hide a real failure.
Runs on free provider tiers; self-hostable.	Some quality checks are lightweight (lexical or embedding based).	Use in teaching distributed systems and AI safety.	Data-protection duties for prompts that are stored.

Table 3.3.5 : SWOT Analysis

3.3.6 Database Design:

Every tenant table carries `developer_id`, and app-scoped data also carries `app_id`, so reads and writes are always filtered by account. The workflow tables (`workflow_instances`, `workflow_events`, `activity_tasks`, `outbox`) form the durable core; the gateway writes `gateway_requests` and `semantic_cache_entry`; pipelines write `trace_step`. Figure 3.3.6 shows the core of the schema.

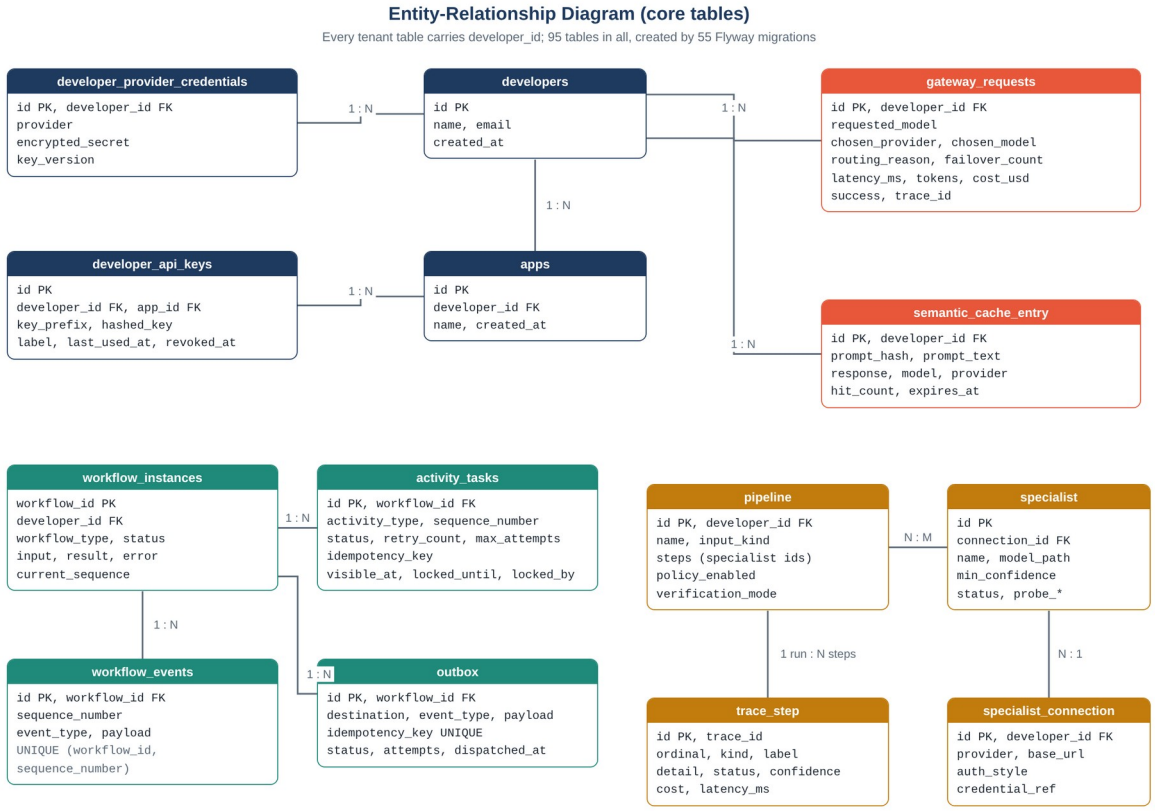


Figure 3.3.6: Entity-Relationship Diagram

3.4 Architecture Diagram

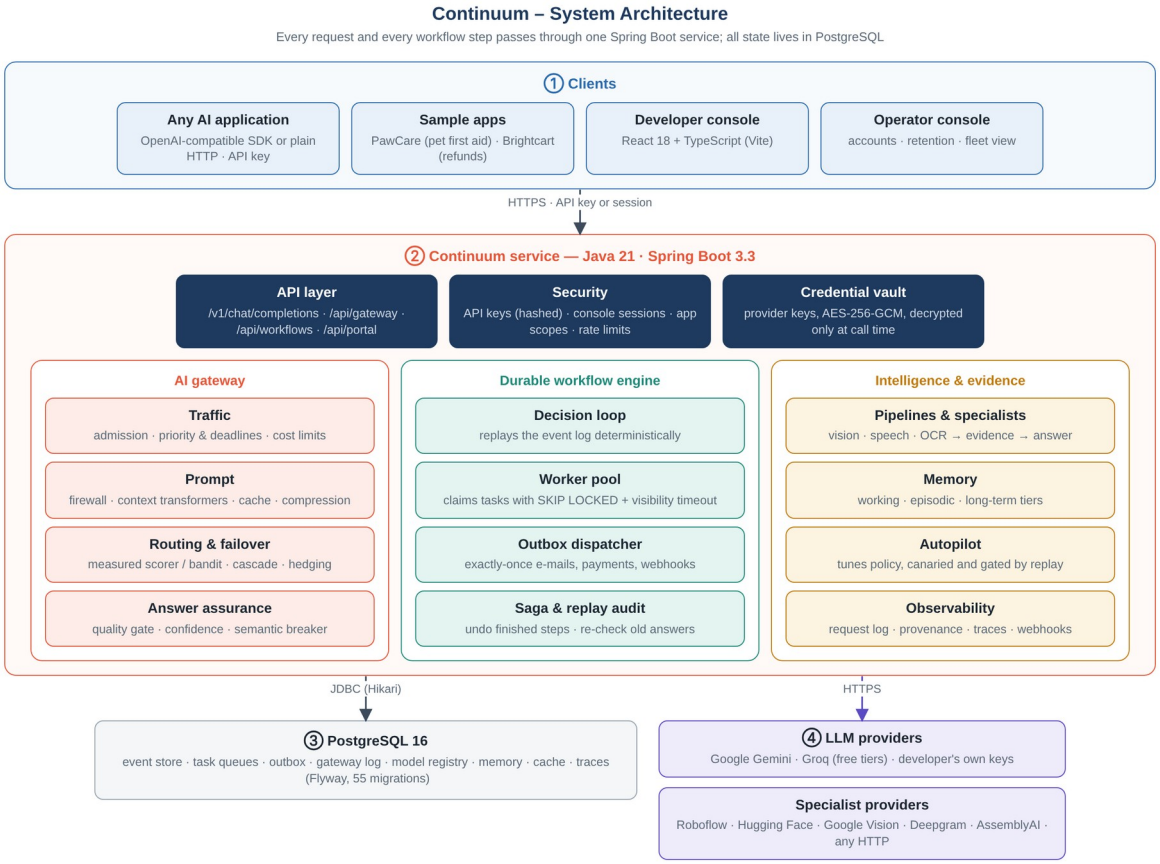


Figure 3.4.1: Proposed System Architecture

The architecture follows the path of a request. Clients — any AI application, the sample apps and the consoles — call one Spring Boot service over HTTPS with an API key or a session. Inside it, the API layer and security components authenticate and scope the call; the AI gateway, the durable workflow engine and the intelligence components do the work; and all state lives in PostgreSQL. The service is the only component that talks to model and specialist providers, using keys from the encrypted credential vault.

Figure 3.4.2 follows one chat request through the gateway; Figure 3.4.3 shows an evidence pipeline; Figure 3.4.4 shows a workflow surviving a crash, and Figure 3.4.5 shows what the event log holds.

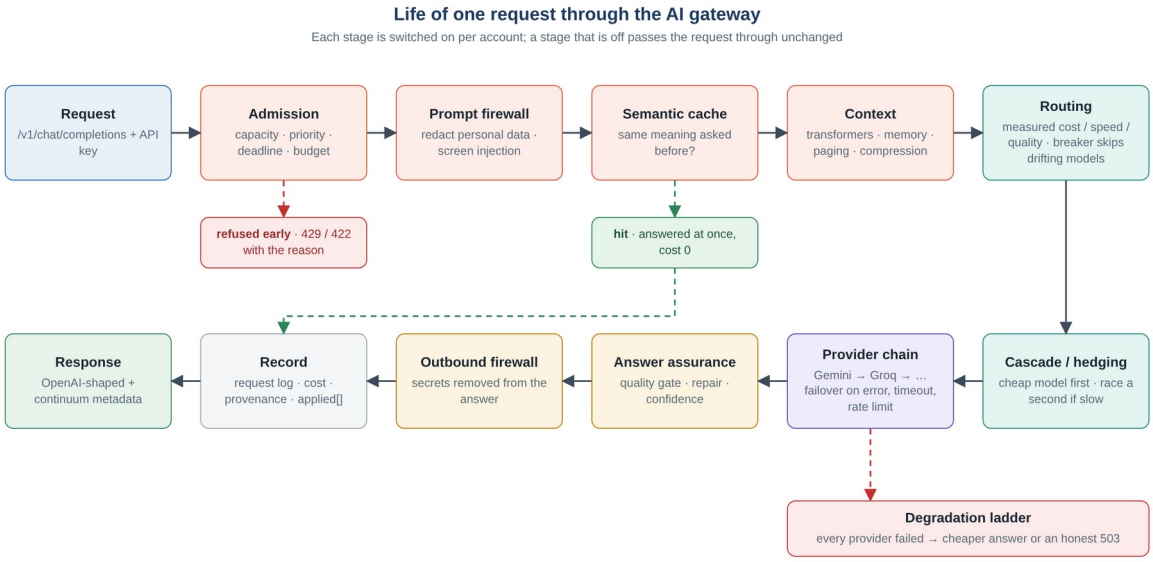


Figure 3.4.2: Request Pipeline of the AI Gateway

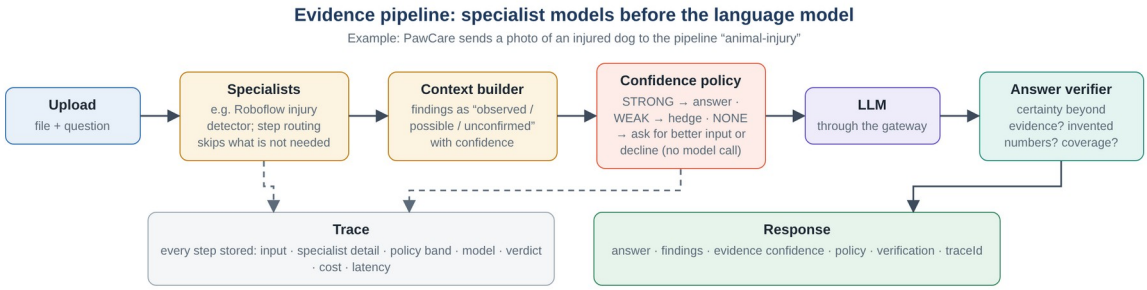


Figure 3.4.3: Evidence Pipeline with Specialist Models

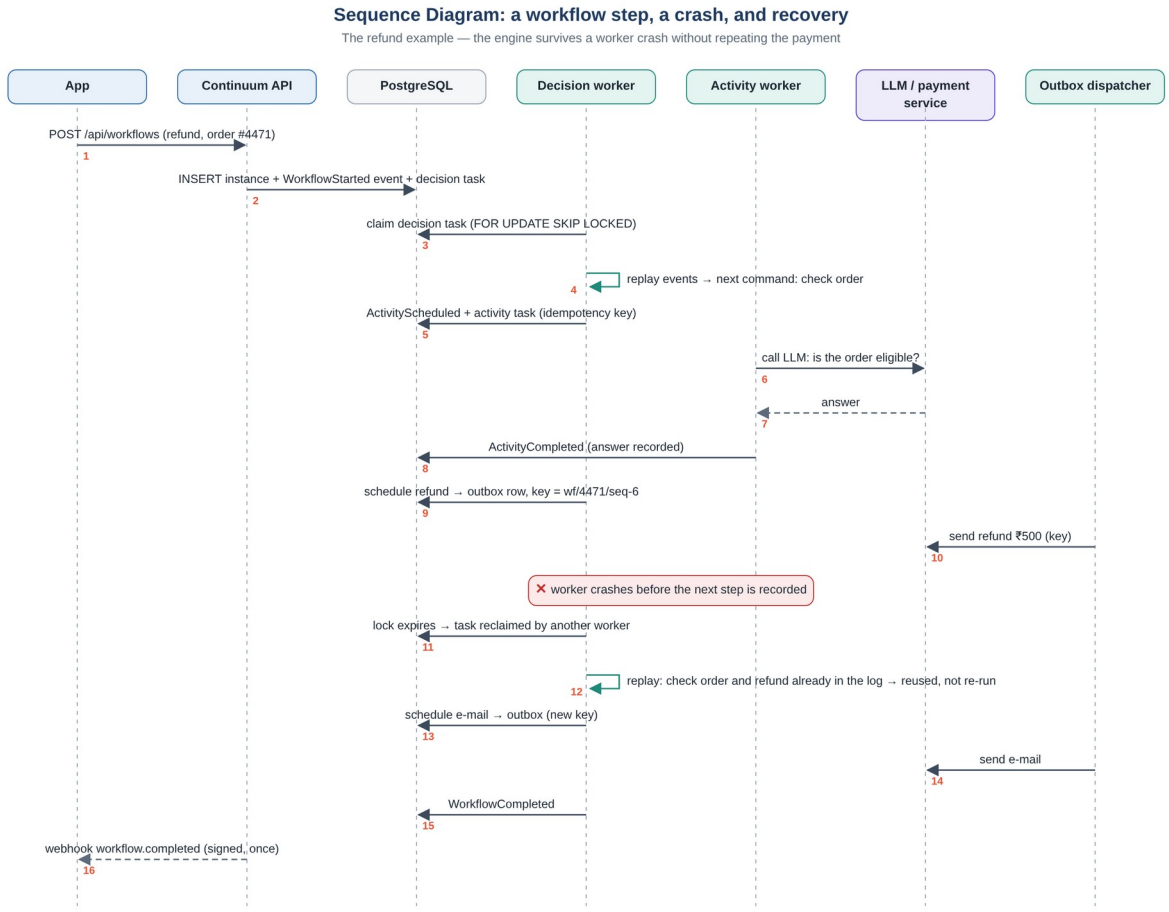


Figure 3.4.4: Sequence Diagram: Workflow Step, Crash and Recovery

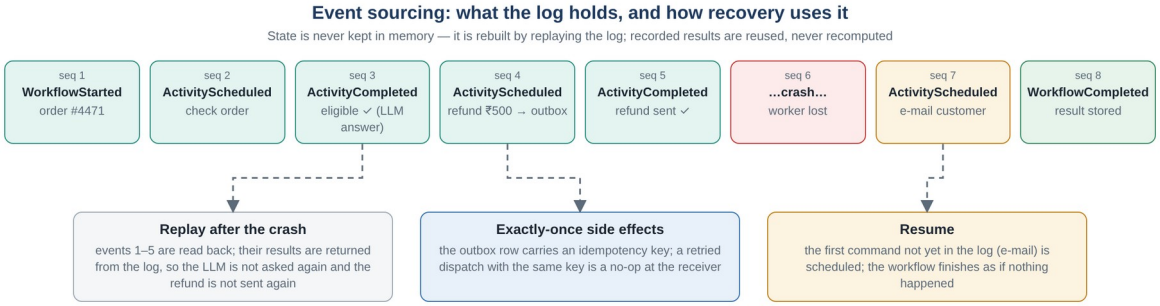


Figure 3.4.5: Event Log and Deterministic Replay

3.5 Flowchart

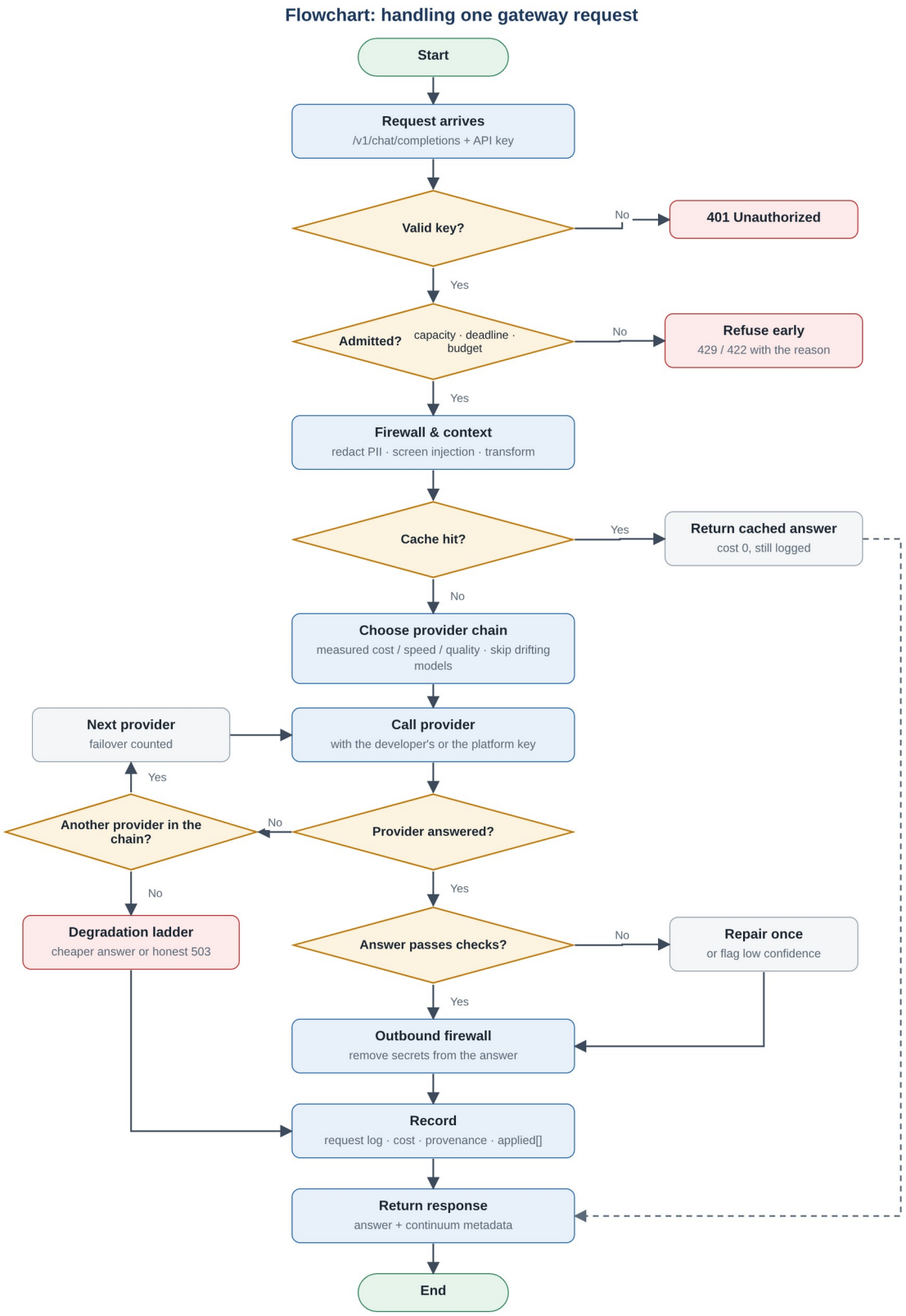


Figure 3.5.1: Proposed System Flowchart (one gateway request)

3.6 Algorithm

3.6.1 Algorithm 1 — Handling a gateway request

Start

1. **Authenticate:**
   Look the API key up by prefix, compare hashes, resolve the account and app; reject an unknown or revoked key with 401.
2. **Admit:**
   Check capacity, priority, deadline and budget. A request that cannot finish in time or within budget is refused with 422 or 429 and the reason.
3. **Guard and prepare:**
   Redact personal data and screen for injection; apply context transformers.
4. **Look up the cache:**
   If an answer to a question with the same meaning is stored and still valid, return it at zero cost and record the hit.
5. **Build context:**
   Add relevant memories, page long histories and compress the prompt within its budget.
6. **Choose the chain:**
   Order (provider, model) options by measured cost, latency and success rate; drop models tripped by the semantic breaker.
7. **Call with failover:**
   Try a cascade or hedge if enabled; otherwise call providers in order. On error, timeout or rate limit, count a failover and try the next. If none is left, step down the degradation ladder.
8. **Assure the answer:**
   Run the quality gate (repair once if it fails), measure confidence if requested, and remove secrets from the answer.
9. **Record and respond:**
   Write the request log, cost, failovers, provenance and the list of features applied; return an OpenAI-shaped response with Continuum metadata.

End

3.6.2 Algorithm 2 — Running a durable workflow

Start

1. **Start:**
   Insert the workflow instance, append a WorkflowStarted event and enqueue a decision task, in one transaction.

2. **Claim:**
   A worker claims a task with FOR UPDATE SKIP LOCKED and sets a visibility timeout (locked_until).
3. **Replay:**
   Load the event log and run the workflow code from the start; every step already recorded returns its recorded result instead of running again.
4. **Decide:**
   The first command not yet in the log is scheduled: an activity task, a timer, or an outbox message with a deterministic idempotency key.
5. **Execute:**
   An activity worker runs the step (for example a model call) with retries and a timeout, and appends ActivityCompleted or ActivityFailed.
6. **Dispatch:**
   The outbox dispatcher sends side effects with their idempotency key and marks them dispatched; a retry with the same key is a no-op.
7. **Recover:**
   If a worker dies, its lock expires and another worker claims the task and continues from step 3.
8. **Finish:**
   Append WorkflowCompleted (or run compensations in reverse order if it failed and the saga is enabled) and send the signed webhook once.

End

3.7 Performance Study

The screenshots in this section were taken from the console running locally with demonstration traffic produced by the project's seed script. A local test provider stood in for Groq, and Gemini had no key in this setup, so the console correctly reports it as unhealthy and routes around it.

3.7.1 API Keys and Providers:

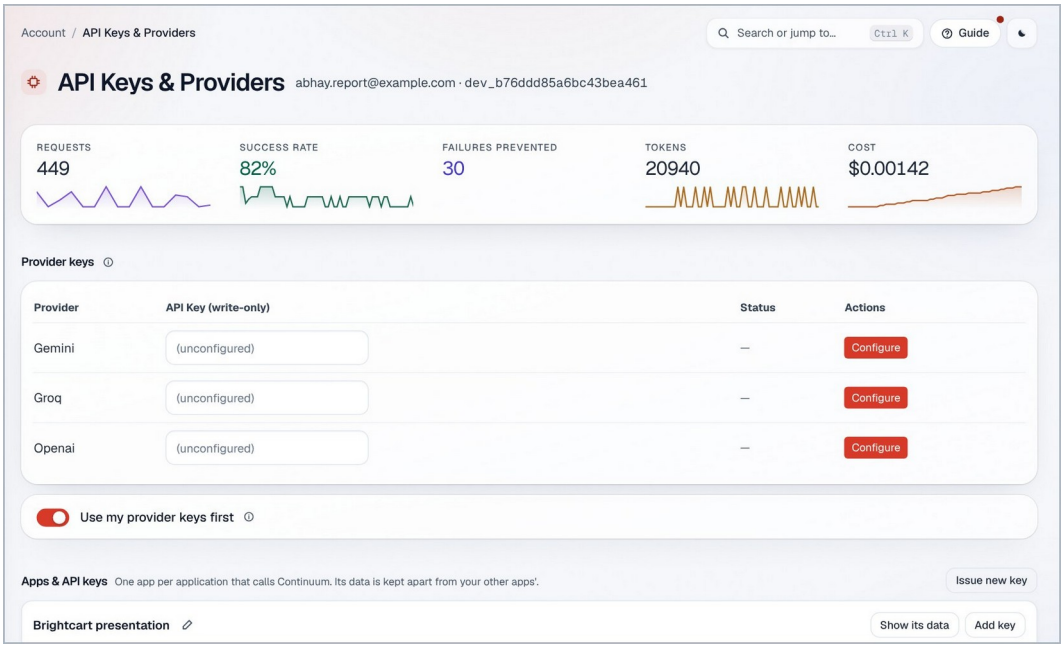


Figure 3.7.1: API Keys, Provider Keys and Webhooks

Integration starts here. The developer issues an API key per application (only a prefix is ever shown again), optionally connects their own provider keys, which are stored encrypted, and registers a webhook endpoint for workflow events.

3.7.2 Command Centre:

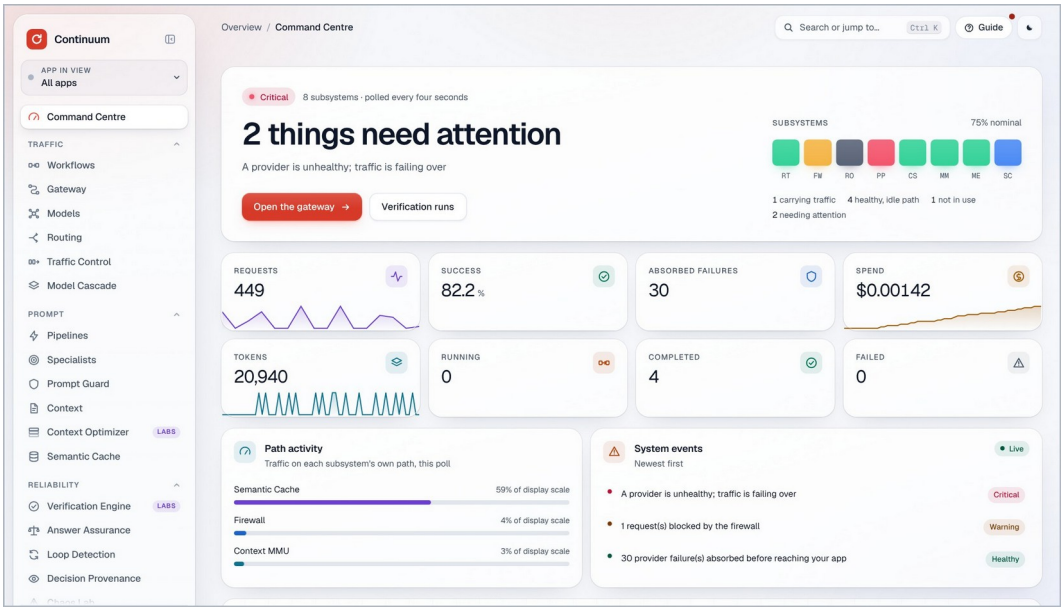


Figure 3.7.2: Command Centre

The command centre summarises the whole account: requests, success rate, failures absorbed, spend and tokens, workflow states, the health of every subsystem and a feed of system events. Here it reports the unconfigured provider and shows that traffic is failing over.

3.7.3 Gateway:

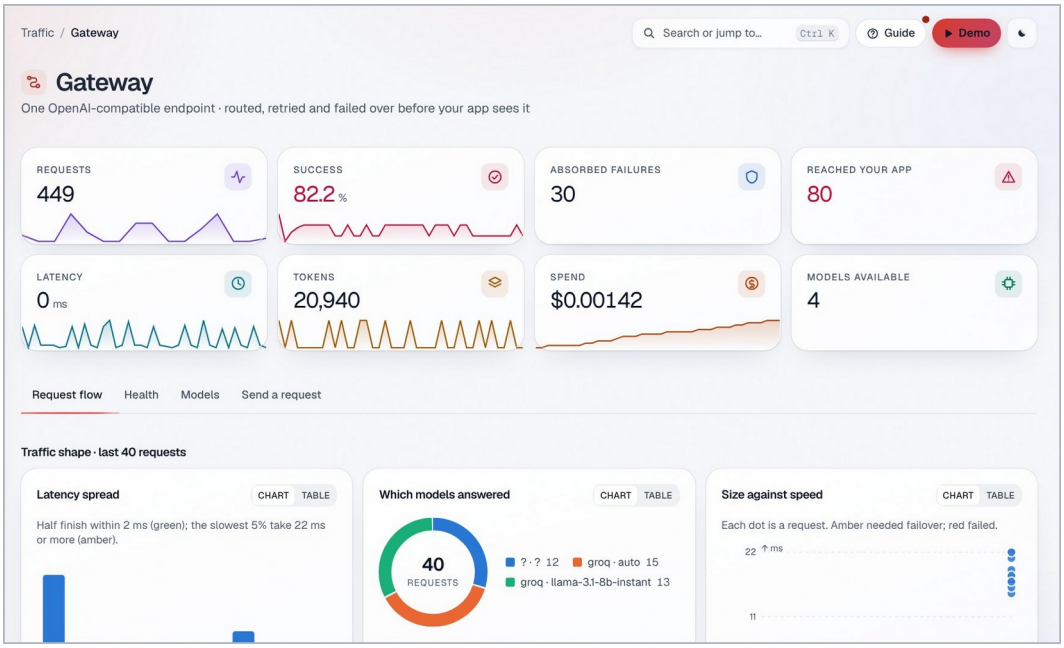


Figure 3.7.3: Gateway: Requests, Latency, Models and Failures

The gateway page shows the OpenAI-compatible endpoint at work: request and success counters, failures absorbed versus failures that reached the application, latency distribution, which models answered, and a live request flow with the reason for each routing decision.

3.7.4 Routing:

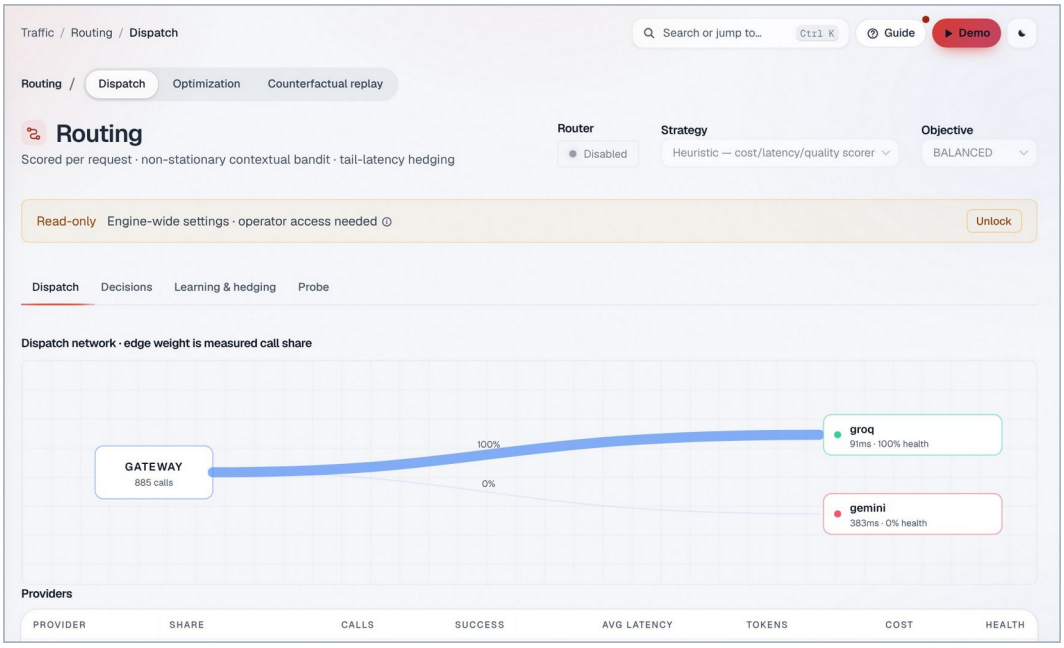


Figure 3.7.4: Routing: Dispatch Network and Provider Statistics

Routing shows how traffic is spread across providers, weighted by measured share, with each provider's calls, success rate, latency, tokens, cost and health. The strategy (heuristic scorer or contextual bandit) and the objective are chosen here.

3.7.5 Workflow Run History:

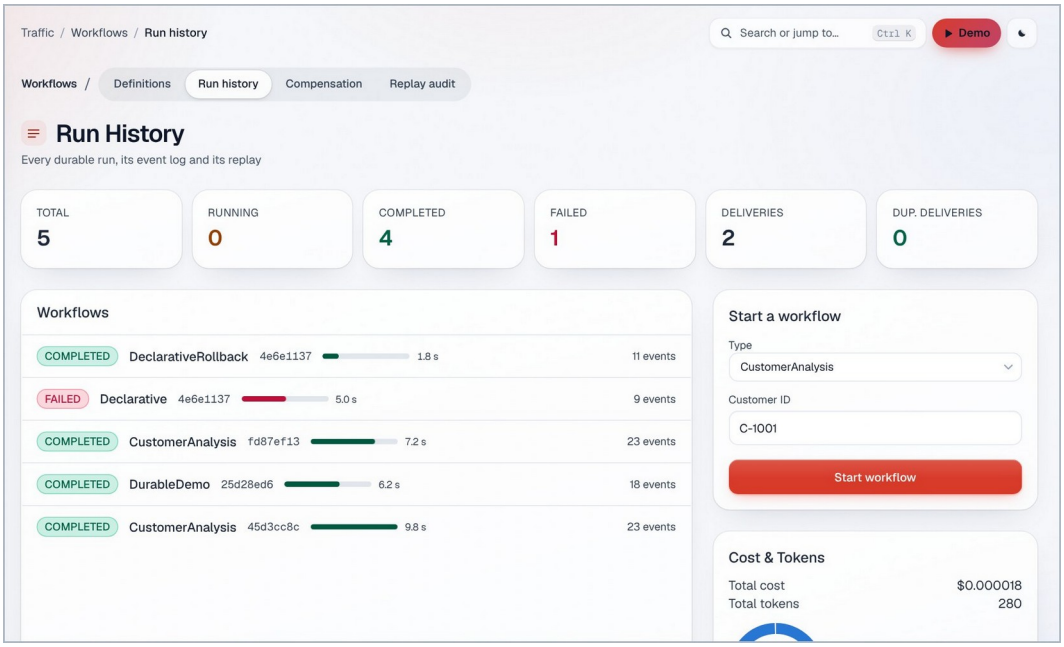


Figure 3.7.5: Durable Workflow Run History

Every durable run is listed with its status, duration and number of recorded events, with counters for completed and failed runs and for outbox deliveries — including duplicate deliveries, which must stay at zero.

3.7.6 Workflow Run Detail:

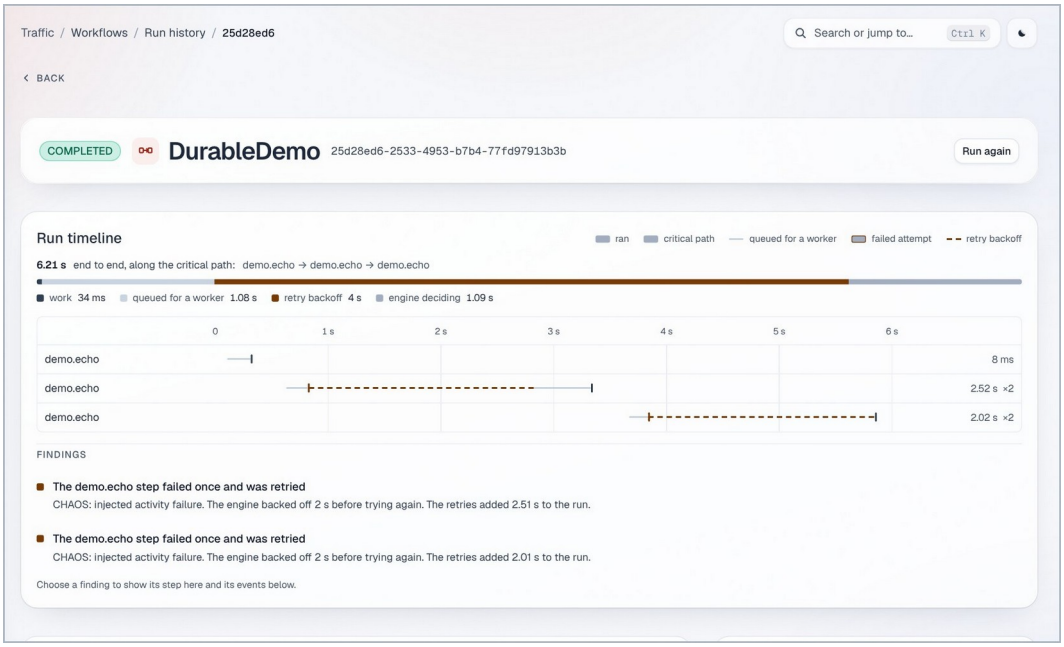


Figure 3.7.6: Run Timeline with Retries and Findings

A run opens as a timeline of its steps along the critical path. In this run the chaos lab failed a step twice; the engine backed off and retried, and the findings explain where the extra time went. Below it, the full event log can be exported.

3.7.7 Saga Compensation:

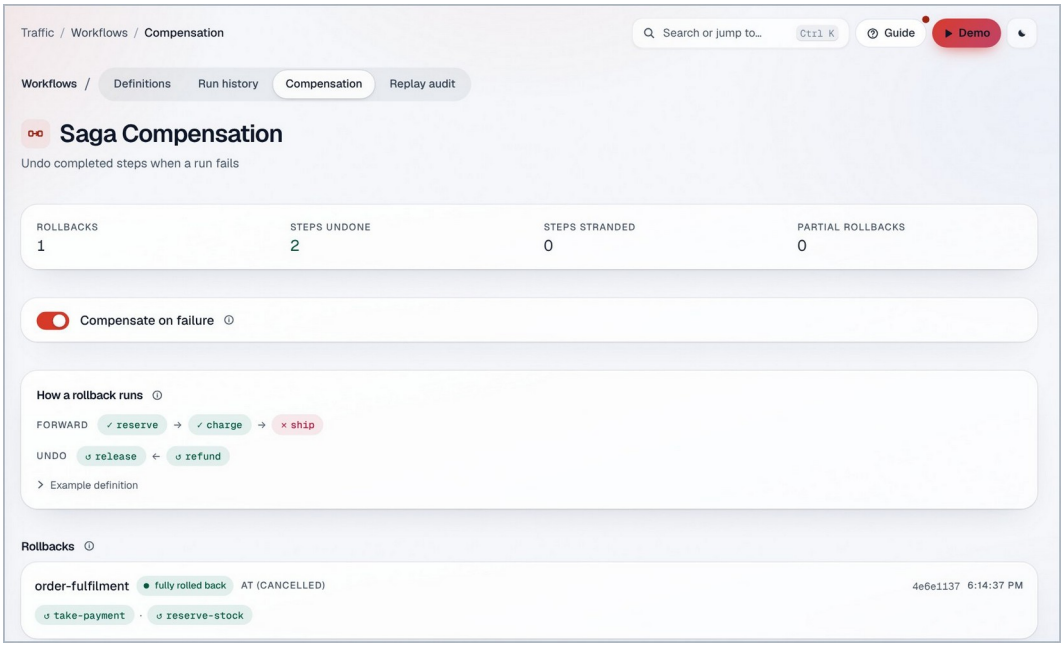


Figure 3.7.7: Saga Compensation of a Cancelled Run

When a run fails or is cancelled, the steps that already completed are undone in reverse order. Here an order-fulfilment run was cancelled after reserving stock and taking payment; Continuum refunded the payment and released the stock.

3.7.8 Semantic Cache:

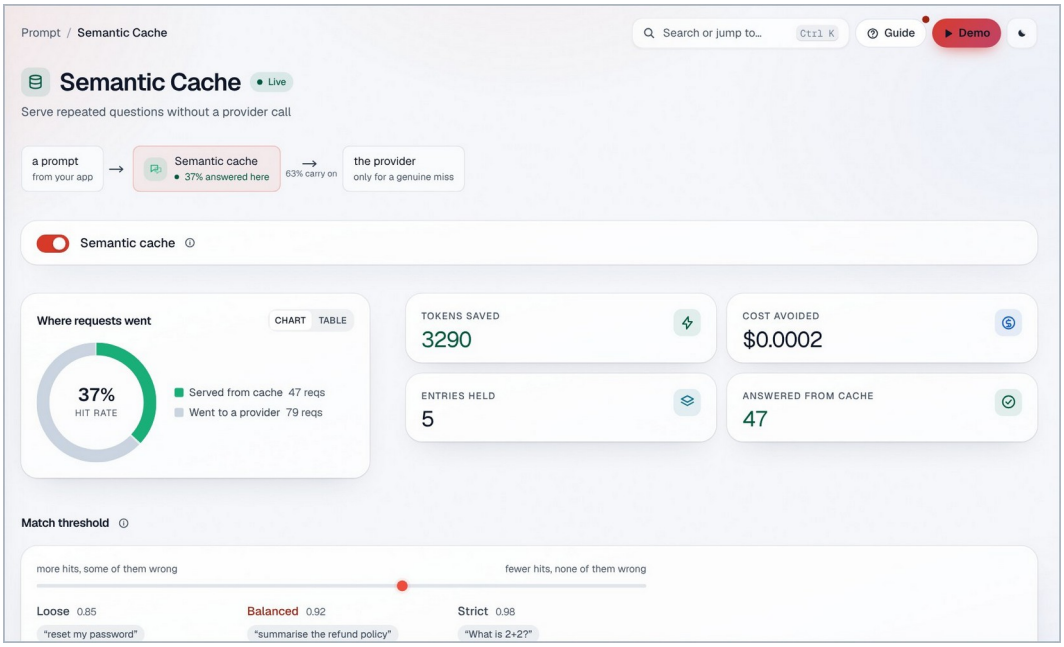


Figure 3.7.8: Semantic Cache: Hit Rate, Tokens Saved and Threshold

The cache page shows how many requests were served from earlier answers, the tokens and cost avoided, and the match threshold, with examples of what counts as the same question at each setting.

3.7.9 Prompt Guard:

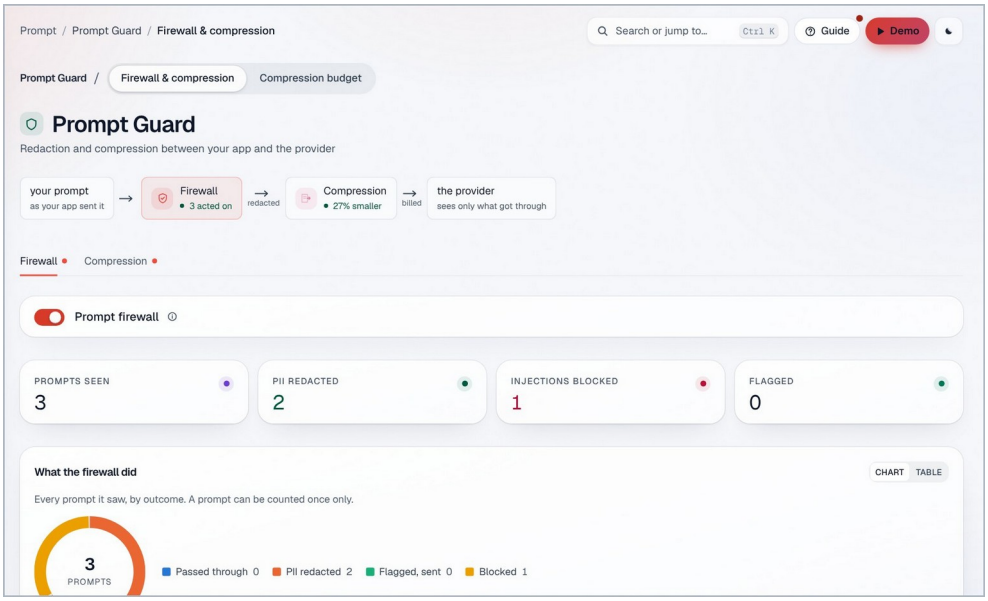


Figure 3.7.9: Prompt Guard: Redaction and Injection Screening

The prompt firewall counts prompts seen, personal data redacted and injections blocked, and shows exactly what the provider receives after redaction: names, e-mail addresses and phone numbers replaced by placeholders, and an injected instruction removed.

3.7.10 Specialists:

Specialist models are added from a catalogue by what they do — read text from a scan, transcribe a recording, detect objects in an image — and are probed before use, so a pipeline never relies on a model that has not been seen to answer.

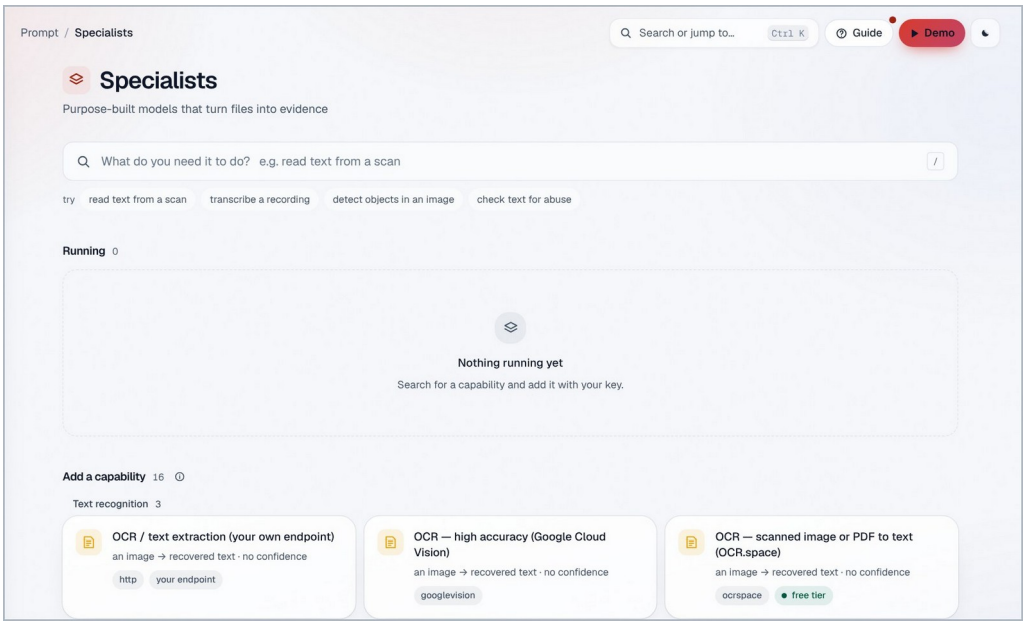


Figure 3.7.10: Specialist Catalogue for Evidence Pipelines

4 | Result and Discussion

4.1 Results

The final system is a working platform that an application adopts by changing one address. Two sample applications were built on it: **PawCare**, a pet first-aid assistant that sends injury photos to an evidence pipeline and remembers each pet's history, and **Brightcart**, a refund desk whose refund runs as a durable workflow with a receipt-reading specialist and webhooks. The figures below show three of the most distinctive features at work: answers verified as a graph of agents, the system surviving deliberately injected model failures, and policy tuned automatically but safely.

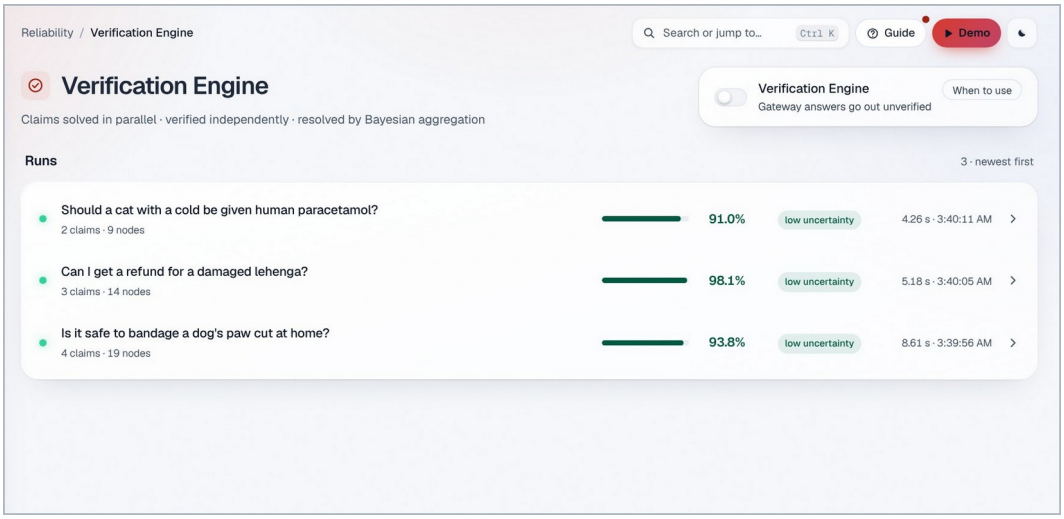


Figure 4.1.1: Verification Engine: Answers Solved and Verified as a Graph

The verification engine splits a question into claims, solves them in parallel, verifies each independently and aggregates the results; every run shows its agreement score and uncertainty.

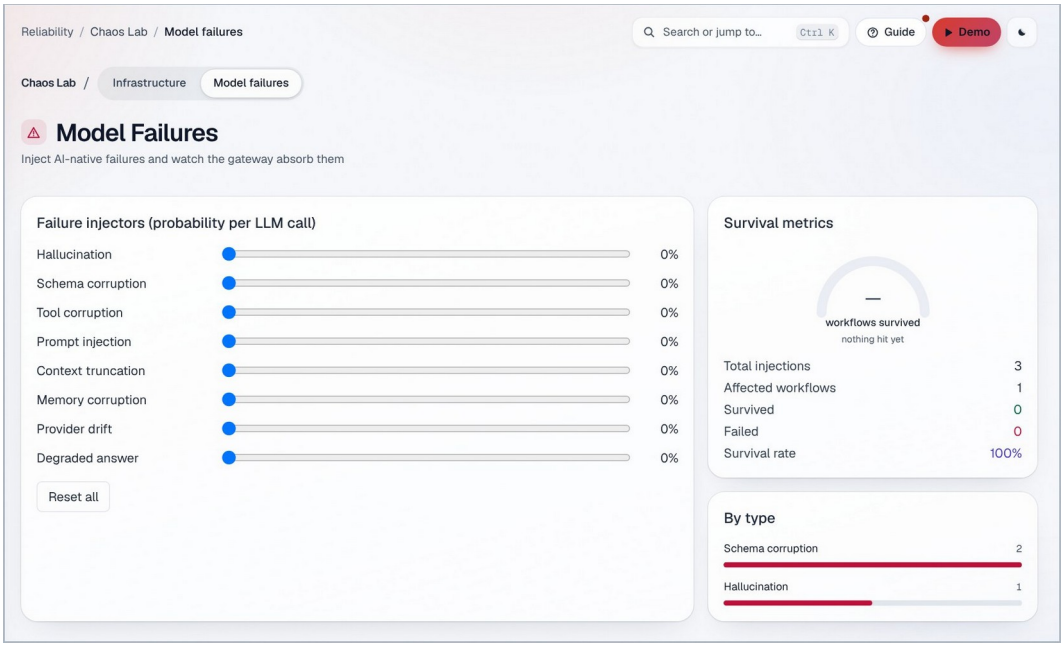


Figure 4.1.2: Chaos Lab: Model Failures Injected and Survived

In the chaos lab, schema corruption and hallucinated decision reversals were injected into model output. The gateway detected and absorbed them: every affected workflow survived, giving a survival rate of 100%.

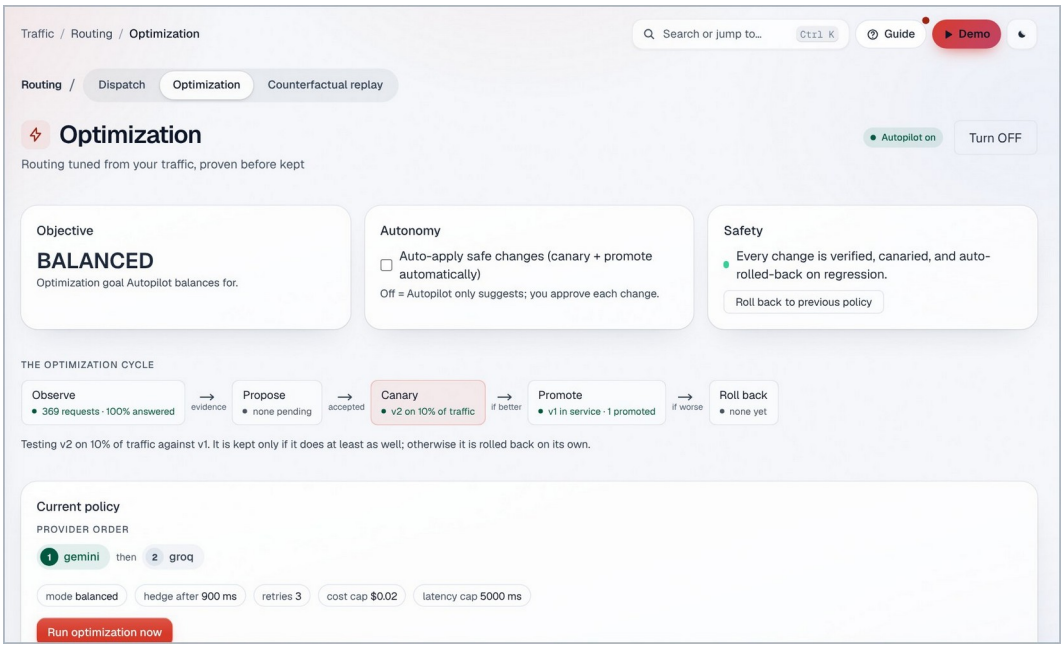


Figure 4.1.3: Autopilot: Policy Optimisation with Canary and Rollback

Autopilot observes traffic, proposes a policy, tests it on a canary share of traffic and promotes it only if it does at least as well; otherwise it rolls back. Each policy bundle is versioned and can be rolled back by hand.

4.2 Reliability and Cost Benchmark

To measure the central claims, the project's benchmark harness sent the same seeded workload — 100 requests drawn from 10 distinct questions, seed 7 — through the gateway over HTTP, once with the semantic cache off and once with it on, while the chaos lab made a fixed share of provider calls fail. The experiment was run twice, with 25% and with 50% of provider calls failing. Both runs were made for this report on a local deployment with a local test provider.

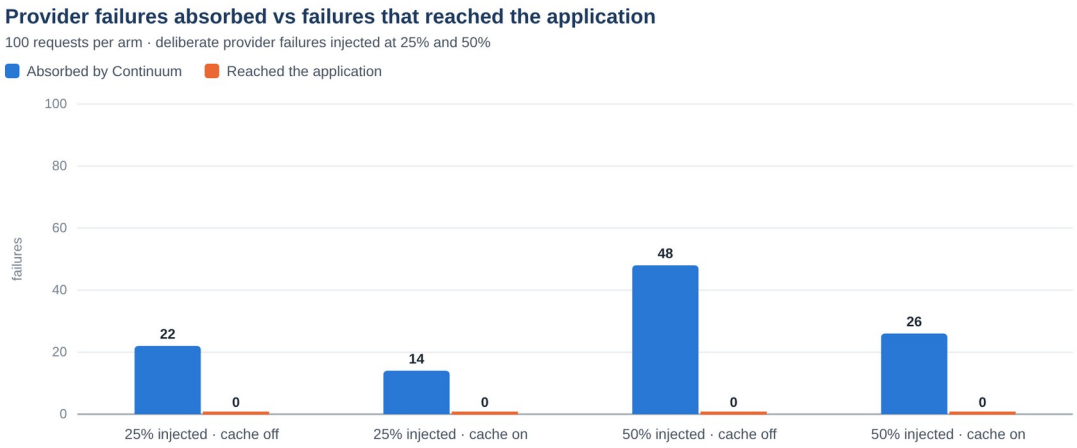


Figure 4.2.1: Provider Failures Absorbed vs Failures that Reached the Application

Measure	25% · off	25% · on	50% · off	50% · on
Requests answered	100	100	100	100
Failures reaching the application	0	0	0	0
Provider failures absorbed	22	14	48	26
Tokens billed	7,000	3,010	3,640	1,890
Spend (USD)	0.000488	0.000198	0.000229	0.000119
Latency p50 (ms)	34.1	13.1	23.3	22.4
Latency p95 (ms)	53.5	41.1	33.4	35.6
Latency p99 (ms)	66.7	48.3	67.8	41.2

Table 4.2.1 : Benchmark Results (100 requests per arm)

Every request was answered in all four arms and **no provider failure reached the application**, even with half of all provider calls failing: Continuum absorbed 22 and 48 failures with the cache off. A direct caller, with no second provider to try, would have seen each of them as an error. With the cache on fewer failures were absorbed simply because cache hits never reach a provider; a few requests were deliberately rate-limited by admission control and still answered.

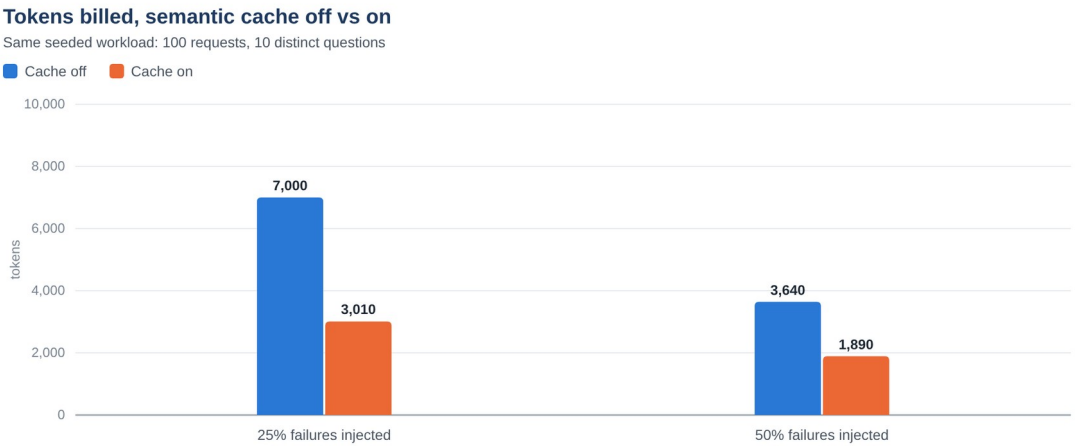


Figure 4.2.2: Tokens Billed with the Semantic Cache Off and On

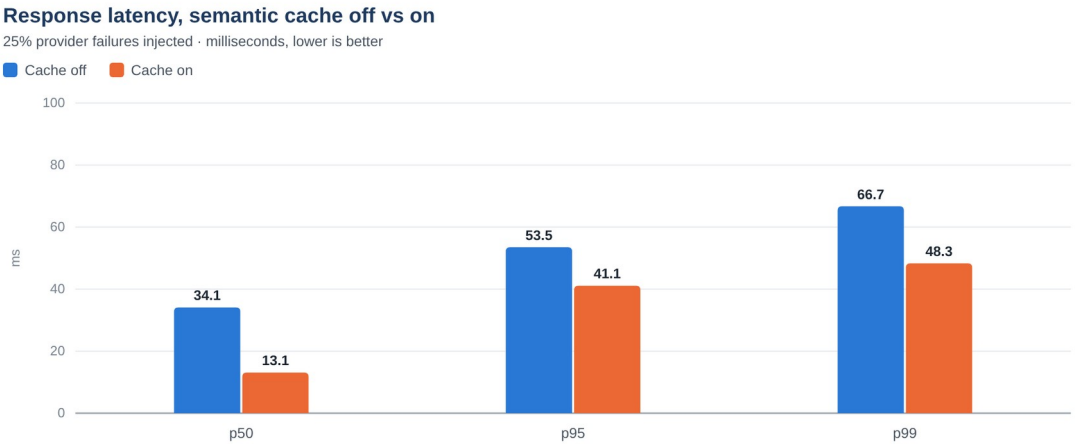


Figure 4.2.3: Response Latency with the Semantic Cache Off and On

The semantic cache reduced tokens billed by **57%** (7,000 to 3,010) in the 25% run and by 48% in the 50% run, and cut median latency from 34.1 ms to 13.1 ms. These savings depend on how often the workload repeats itself — this one repeats by construction — so they show the mechanism working rather than promising the same saving on any traffic. Absolute latencies are low because the provider was local; with a hosted provider the gap between a cache hit and a provider call is much larger.

4.3 Self-healing under Code Change

A durable workflow must keep replaying correctly even when its code is changed while runs are in flight. The project's paradox-healing benchmark deploys a random structural change (a step inserted, deleted or reordered) under a running workflow in each of 300 seeded trials and checks the invariants. It was re-run for this report as part of the unit-test suite.

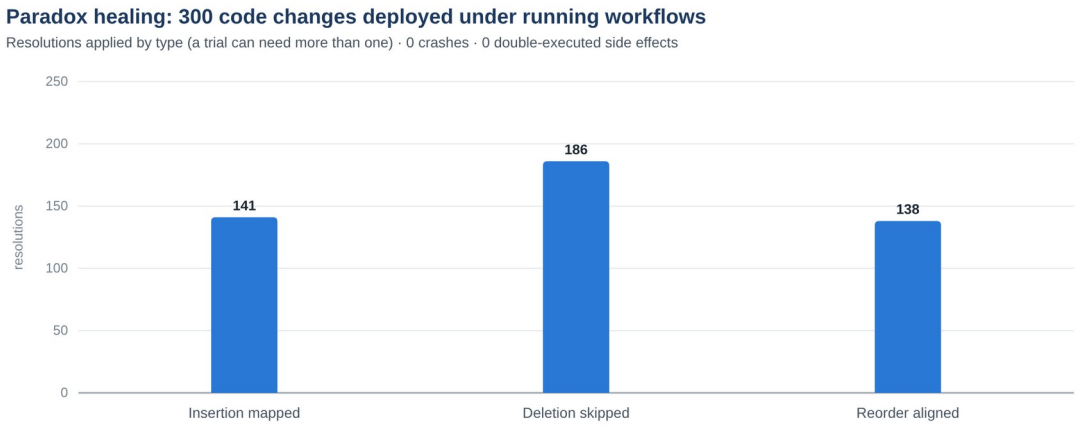


Figure 4.3.1: Resolutions Applied over 300 Code Changes

Measure	Result
Code changes deployed under running workflows	300
Crashes	0
Double-executed side effects	0
Double-executed activities	0
Trials that diverged structurally and were healed	266
Resolutions by type	insertion mapped 141 · deletion skipped 186 · reorder aligned 138

Table 4.3.1 : Paradox-Healing Benchmark (300 seeded trials)

4.4 Other Measurements

Further measurements recorded during development, each from a local run, are summarised below. They describe single runs and are reported as observations rather than general claims.

Feature	Setting	Observation
Model cascade	12 requests of mixed difficulty, two model tiers	3 escalated (25%); spend $0.000965 vs $0.001120 for the strong model alone — 14% saved; 0 wasted escalations
Worker concurrency	25 concurrent workflows	peak of 10 concurrent activities, bounded by the batch size (1 before the change)
Priority & deadlines	provider taking ≈ 9 ms	a 1 ms deadline is refused at once with HTTP 422; a 60 s deadline is answered normally
Compression budget	asked to keep 85% of an instruction block	kept 75% — revealed a bug (stop-words always dropped), which was then fixed

Table 4.4.1 : Measurements Recorded during Development

4.5 Testing

Each part of the system has automated tests, run repeatedly during development. The suites were run again for this report.

Component	Framework	Tests	What is covered
Service (unit)	JUnit 5, Mockito	881	engine replay and recovery, routing, failover, cache, cascade, firewall, confidence, quality gate, specialists, pipelines, security
Service (integration)	JUnit 5, Spring Boot Test, PostgreSQL	75	migrations, app scoping, the program API and gateway end to end against a real database (6 need live provider keys and were skipped)
Console	Vitest	33	navigation map, feature registry, content-security policy, formatting
Benchmarks	Python harness, JUnit	2	reliability and cost (Section 4.2); self-healing under code change (Section 4.3)

Table 4.5.1 : Automated Test Suites

All 881 unit tests passed. One of them, which checks a background confirmation thread in the model catalogue, failed once when the whole suite ran under load and passed when re-run; it is timing-sensitive and is noted for hardening. All 69 integration tests that could run passed, and all 33 console tests passed.

4.6 Discussion

The results show three things. First, the reliability claim holds under stress: with up to half of all provider calls failing, every request was answered and none of the failures reached the application, because the gateway counted each failure and moved on to the next provider. Second, the durable engine’s guarantees hold under change as well as under crashes: 300 code changes deployed under running workflows produced no crash and no repeated side effect. Third, the cost features pay for themselves when traffic repeats, and the console makes the saving visible per request.

The limits are also clear. The benchmark used a local test provider, so its latencies are not those of a hosted model; the semantic checks are deliberately lightweight, so they flag doubt rather than prove correctness; and the system runs as one service on one database, which is simple to operate but scales vertically first. Because every feature is switched per account and off by default, each can be adopted, measured and removed independently.

5 | Conclusion

Continuum is a self-hostable platform that makes applications built on large language models reliable, safe and affordable without changing how they are written. It combines a durable execution engine — event-sourced workflows with deterministic replay, safe task claiming, an exactly-once outbox and saga compensation — with an OpenAI-compatible gateway that routes, fails over, caches, cascades, guards and checks every model call, and records why each decision was taken. Evidence pipelines put specialist models before the language model so that answers rest on measured findings.

The evaluation supports the main claims with reproducible measurements: on a seeded workload with 25% and 50% of provider calls made to fail, all requests were answered and no failure reached the application; the semantic cache cut tokens billed by 57% and median latency by more than half on repeated traffic; and 300 code changes deployed under running workflows caused no crash and no repeated side effect. The platform is covered by 881 unit, 75 integration and 33 console tests.

Future scope. The most valuable next steps are to run the benchmark against hosted providers under real load; to make the gateway horizontally scalable with a shared rate limiter; to stream tool calls through the durable engine so that agent frameworks get crash safety for free; to strengthen answer verification with stronger judge models where the cost is justified; to add more specialist providers and Indian-language sample applications; and to offer Continuum as a hosted, multi-tenant service for student and start-up teams.

Nevertheless, no layer around a model can make the model itself correct. Continuum’s contribution is to make the uncertainty visible, the failures harmless and the costs controlled, so that the people building AI features can trust what their application does even when the model or its provider does not behave.

References

[1] Temporal Technologies, “Temporal Documentation: Durable Execution.” [Online]. Available: https://docs.temporal.io/

[2] M. Fowler, “Event Sourcing,” martinfowler.com, Dec. 2005. [Online]. Available: https://martinfowler.com/eaaDev/EventSourcing.html

[3] H. Garcia-Molina and K. Salem, “Sagas,” *Proc. ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987.

[4] C. Richardson, *Microservices Patterns*, Manning Publications, 2018 (Transactional Outbox, Idempotent Consumer).

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[6] L. Kuhn, Y. Gal and S. Farquhar, “Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation,” *Proc. ICLR*, 2023.

[7] S. Farquhar, J. Kossen, L. Kuhn and Y. Gal, “Detecting hallucinations in large language models using semantic entropy,” *Nature*, vol. 630, pp. 625–630, 2024.

[8] L. Chen, M. Zaharia and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv: 2305.05176, 2023.

[9] L. Chen, M. Zaharia and J. Zou, “How Is ChatGPT’s Behavior Changing over Time?,” arXiv: 2307.09009, 2023.

[10] J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[12] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez et al., “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv: 2406.18665, 2024.

[13] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv: 2310.08560, 2023.

[14] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang and L. Qiu, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” *Proc. EMNLP*, 2023.

[15] F. Bang, “GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings,” *Proc. Workshop for NLP Open Source Software (NLP-OSS)*, 2023.

[16] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds and C. Rosenthal, “Chaos Engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[17] L. Li, W. Chu, J. Langford and R. E. Schapire, “A Contextual-Bandit Approach to Personalized News Article Recommendation,” *Proc. WWW*, pp. 661–670, 2010.

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[19] The PostgreSQL Global Development Group, “PostgreSQL 16 Documentation: SELECT — The Locking Clause (SKIP LOCKED).” [Online]. Available: https://www.postgresql.org/docs/16/sql-select.html

[20] Spring Boot Reference Documentation, VMware / Broadcom. [Online]. Available: https://spring.io/projects/spring-boot

[21] Meta Open Source, “React Documentation.” [Online]. Available: https://react.dev/

[22] OpenAI, “API Reference: Chat Completions.” [Online]. Available: https://platform.openai.com/docs/api-reference/chat

[23] Docker Inc., “Docker and Docker Compose Documentation.” [Online]. Available: https://docs.docker.com/